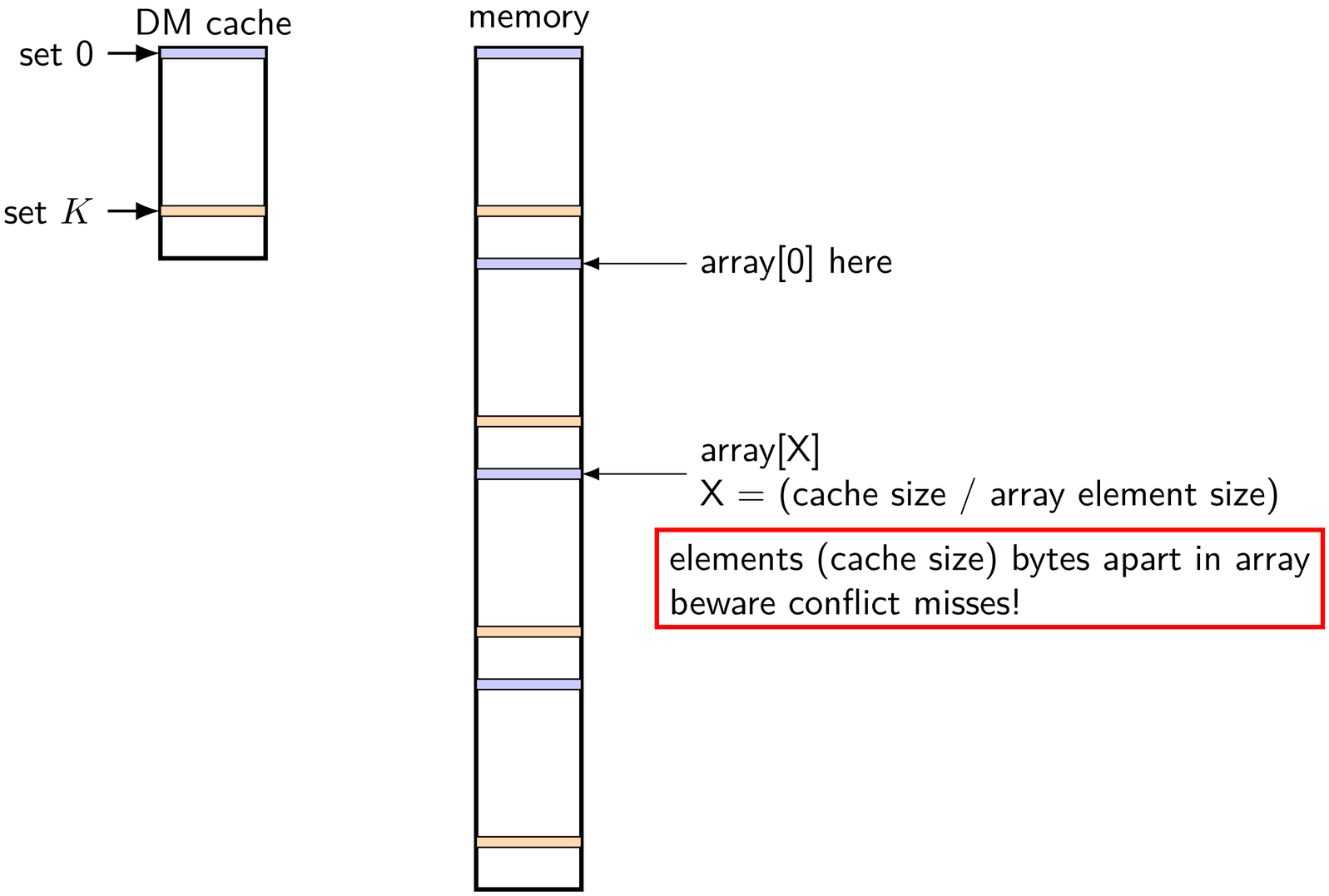
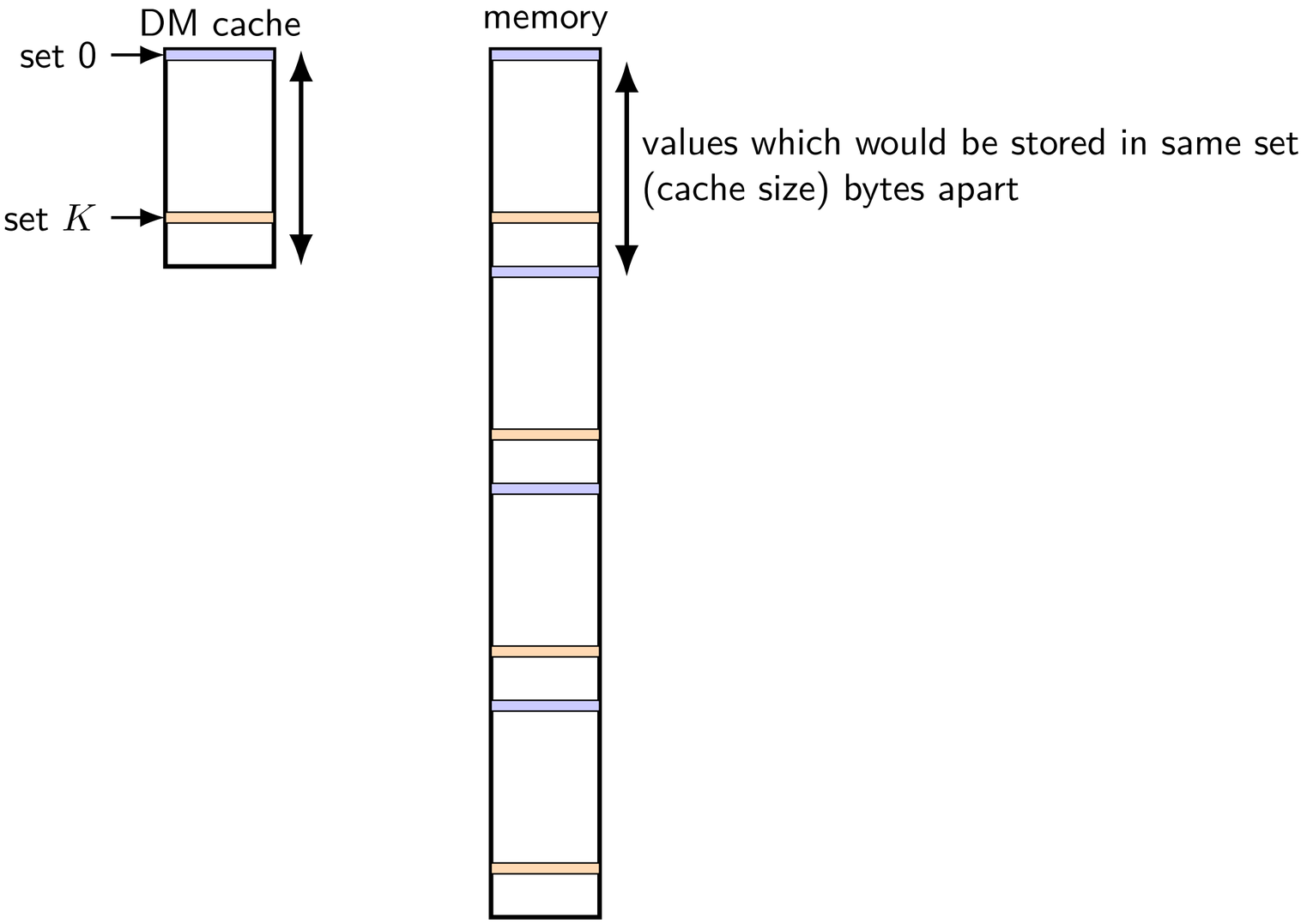
mapping of sets to memory (direct-mapped)

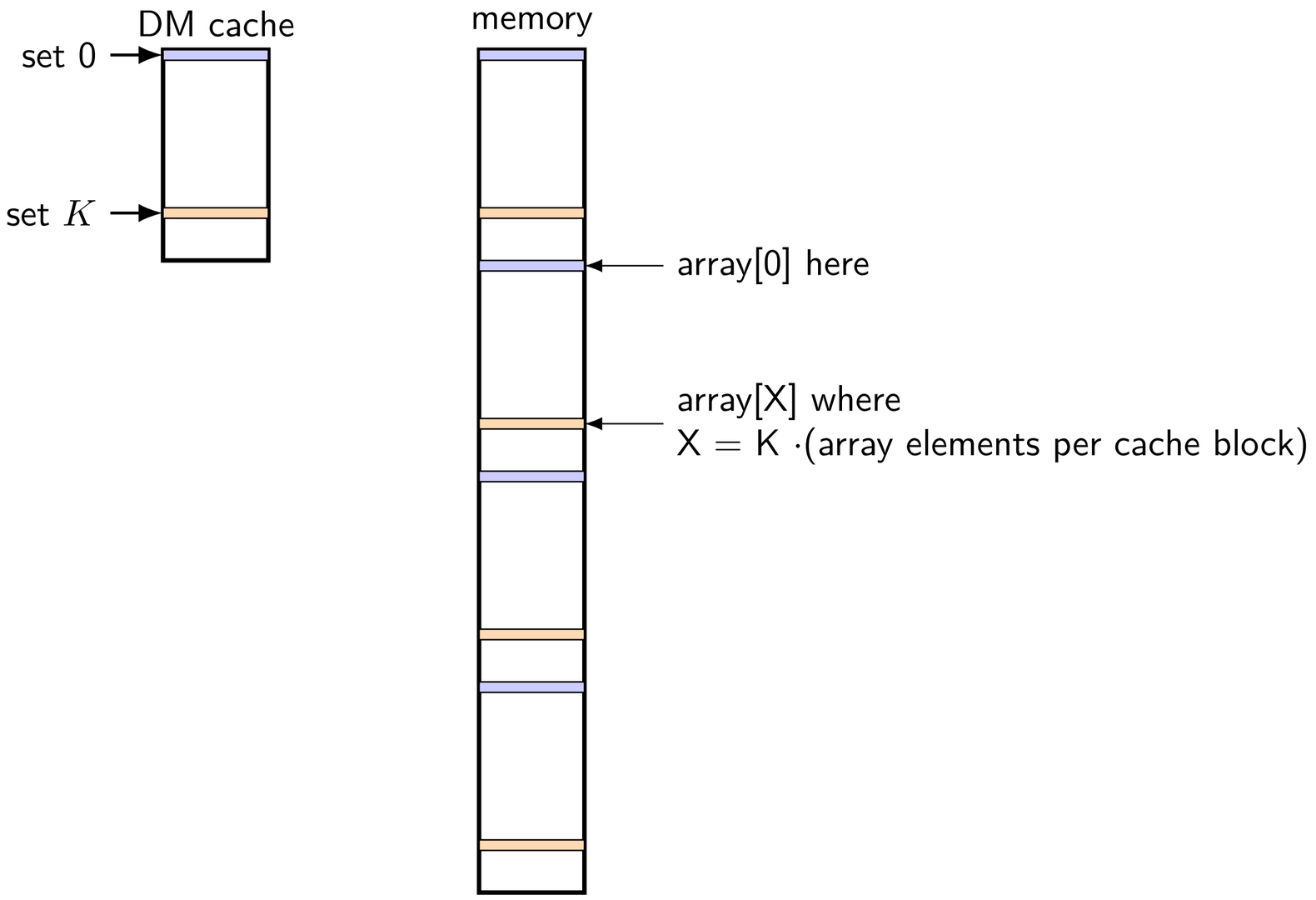


mapping of sets to memory (direct-mapped)

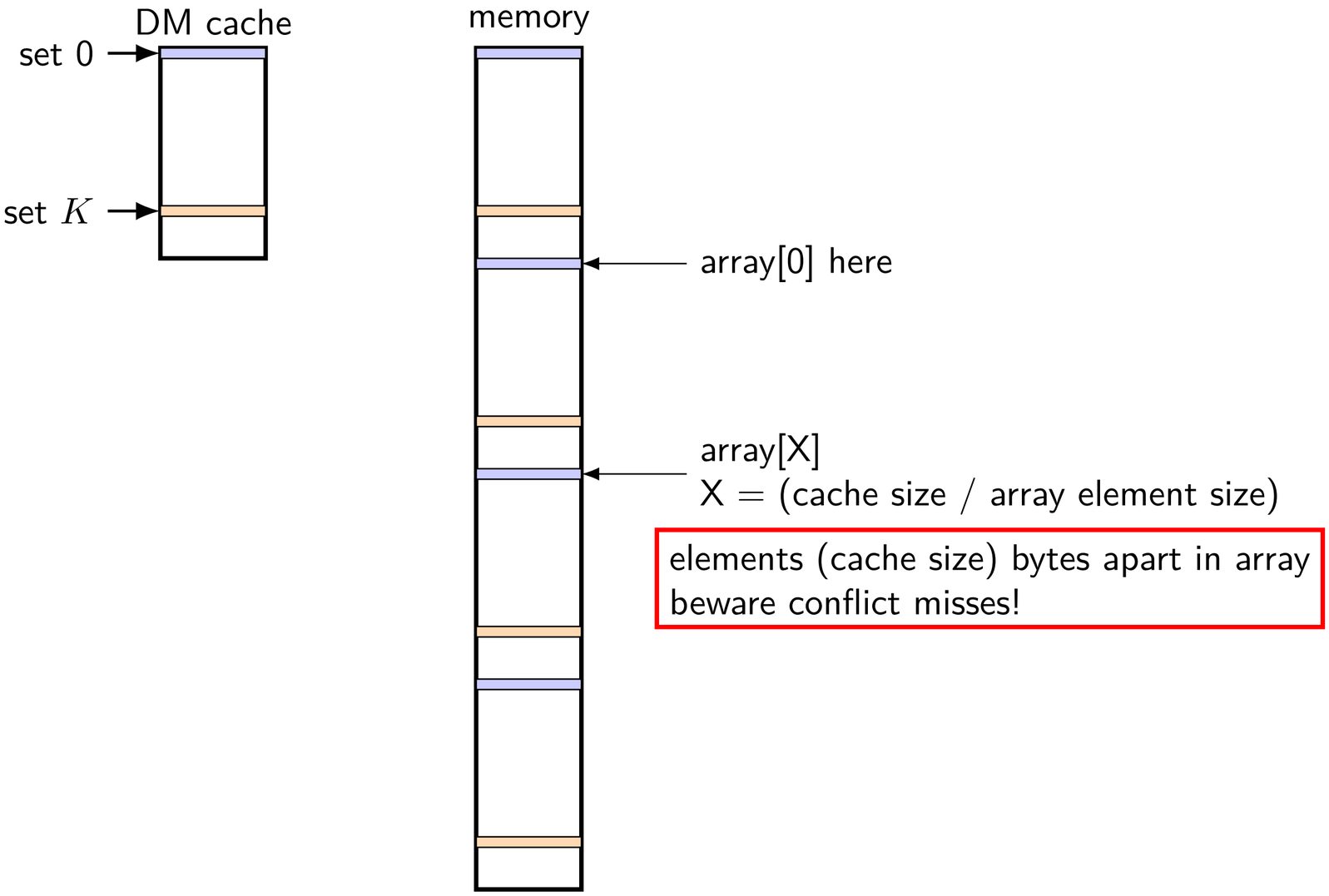
mapping of sets to memory (direct-mapped)



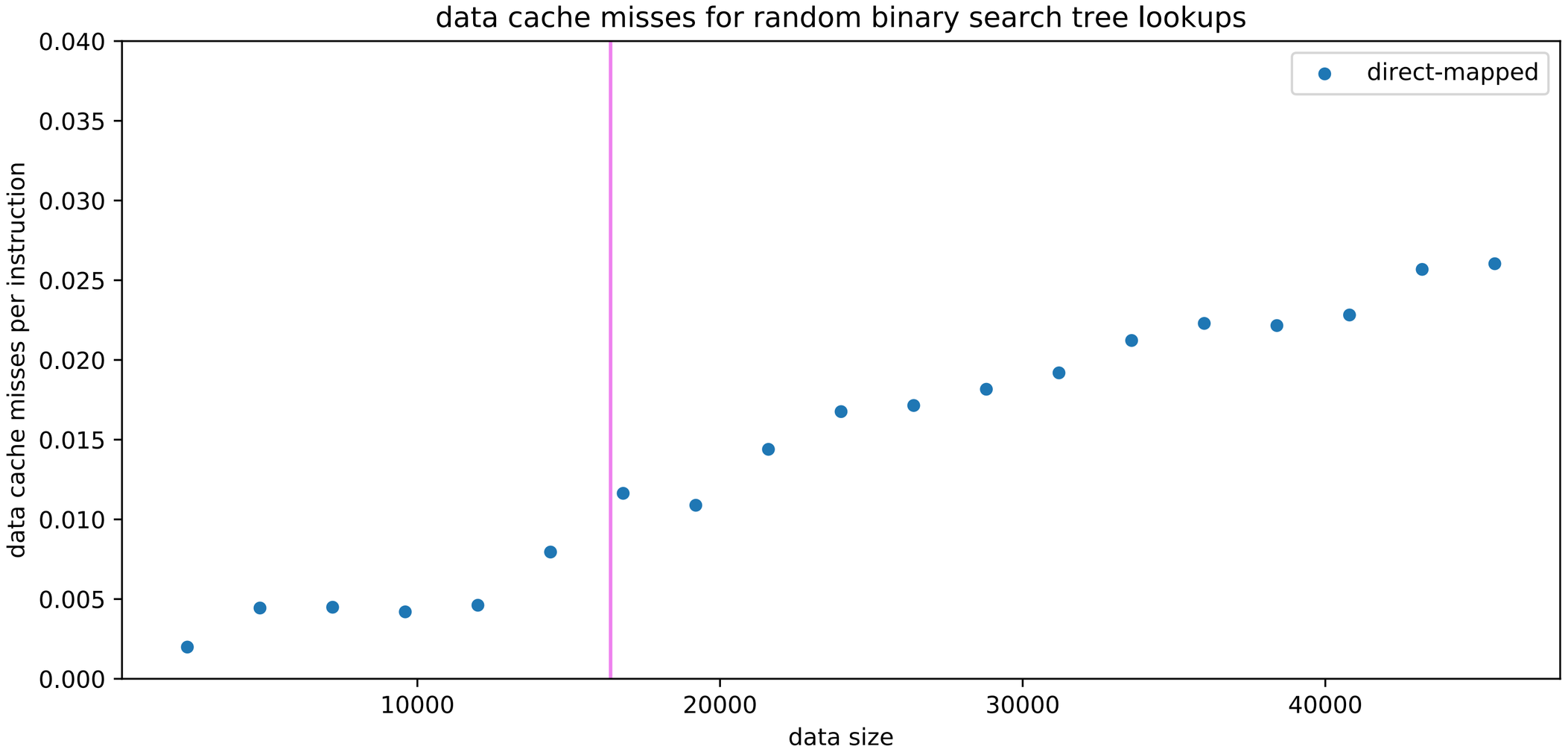
mapping of sets to memory (direct-mapped)



mapping of sets to memory (direct-mapped)

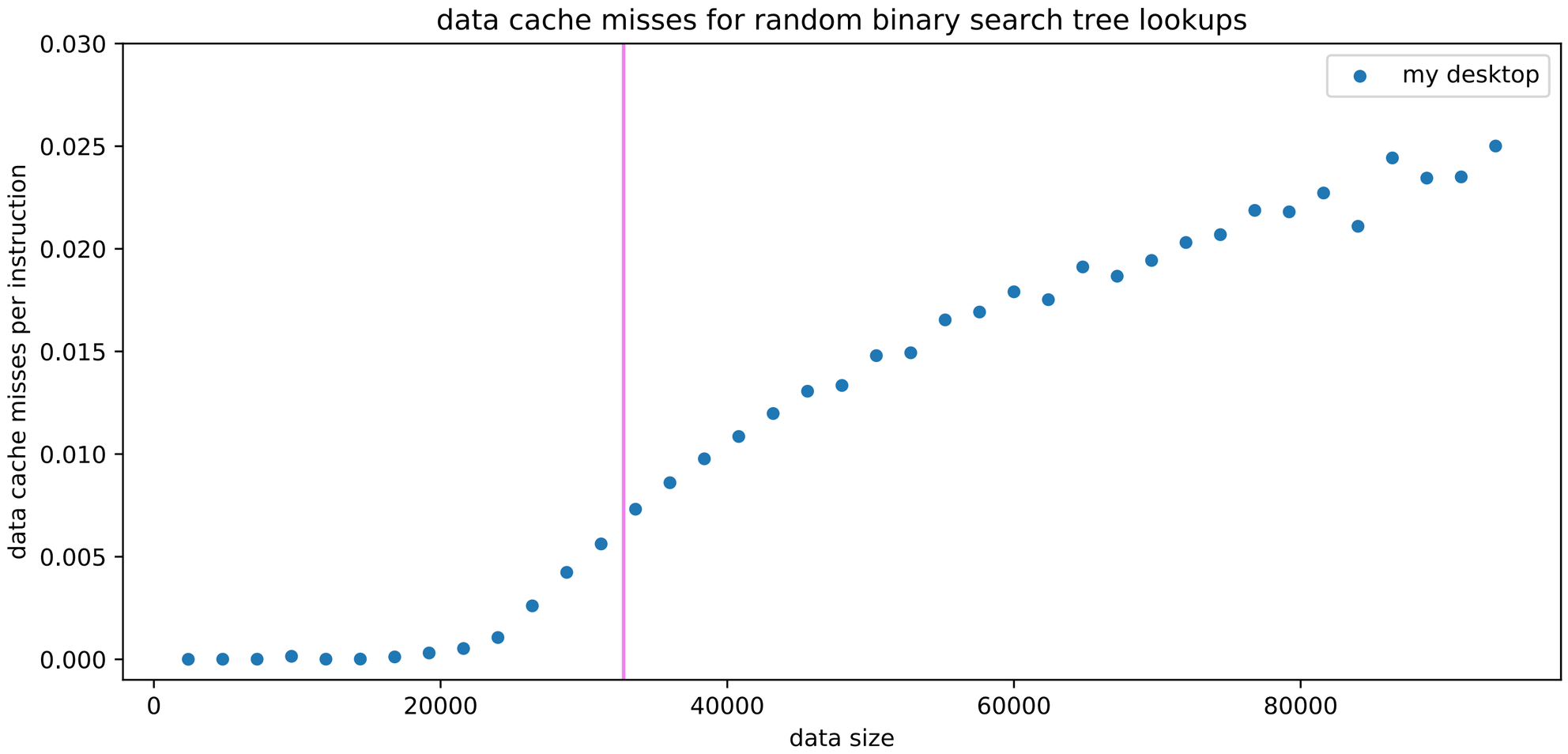


simulated misses: BST lookups



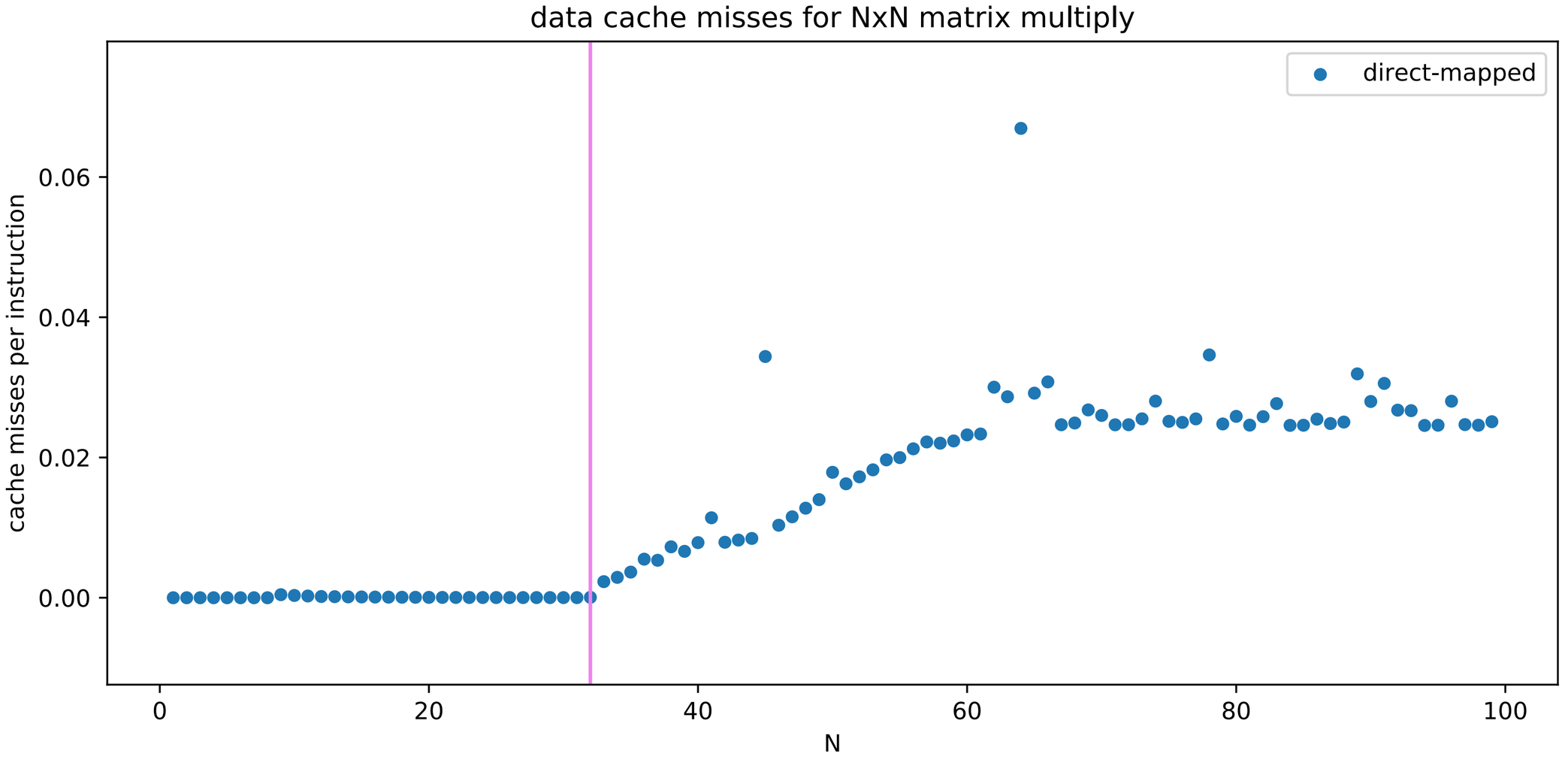
simulated 16KB direct-mapped cache; excluding BST setup

actual misses: BST lookups



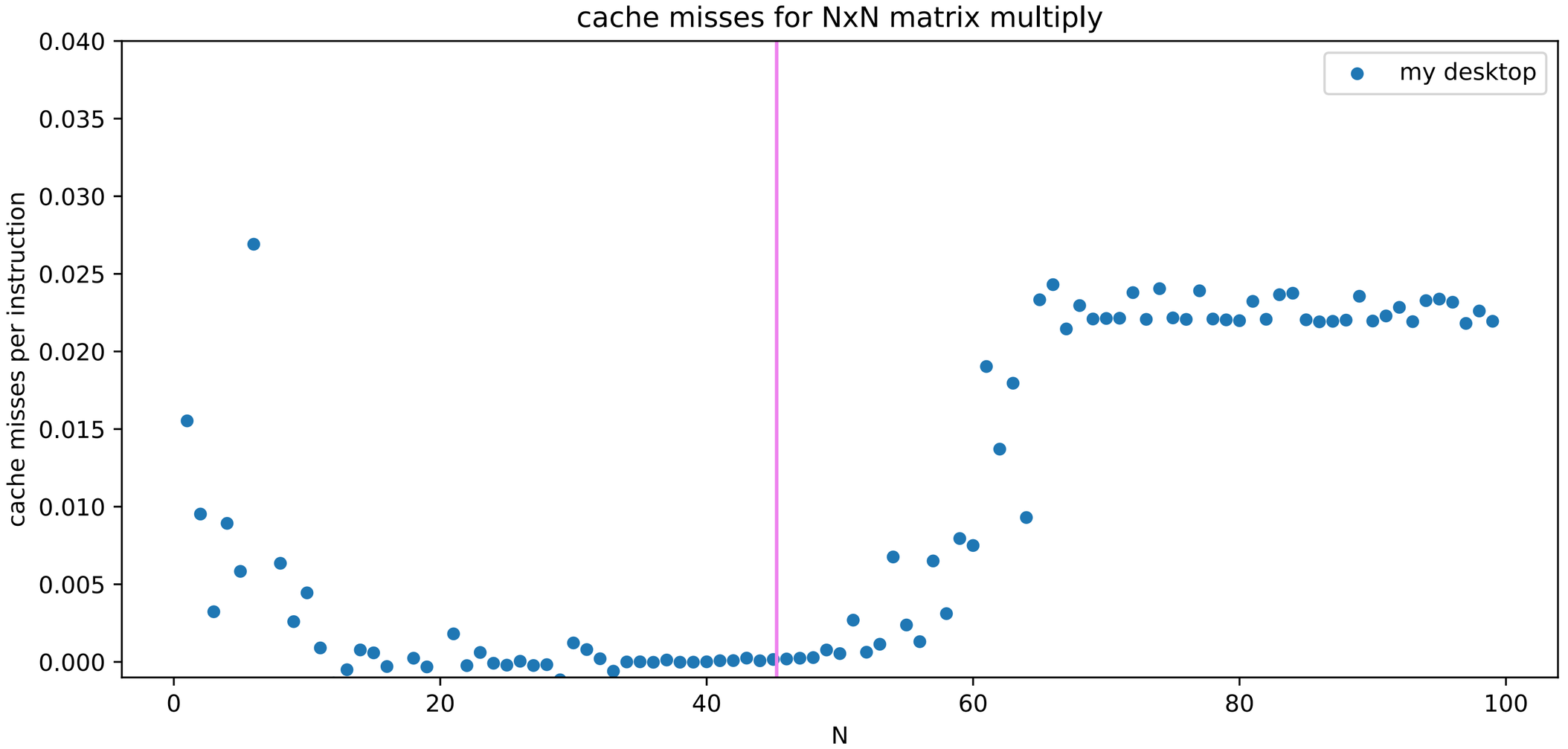
actual 32KB more complex cache (only one set of measurements + other things on the machine + excluding initial load)

simulated misses: matrix multiplies



simulated 16KB direct-mapped data cache; excluding initial load

actual misses: matrix multiplies



actual 32KB more complex cache; excluding initial matrix load

adding associativity

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	0			0		
1	0			0		

multiple places to put values with same index
avoid misses from two active values using same set
("conflict misses")

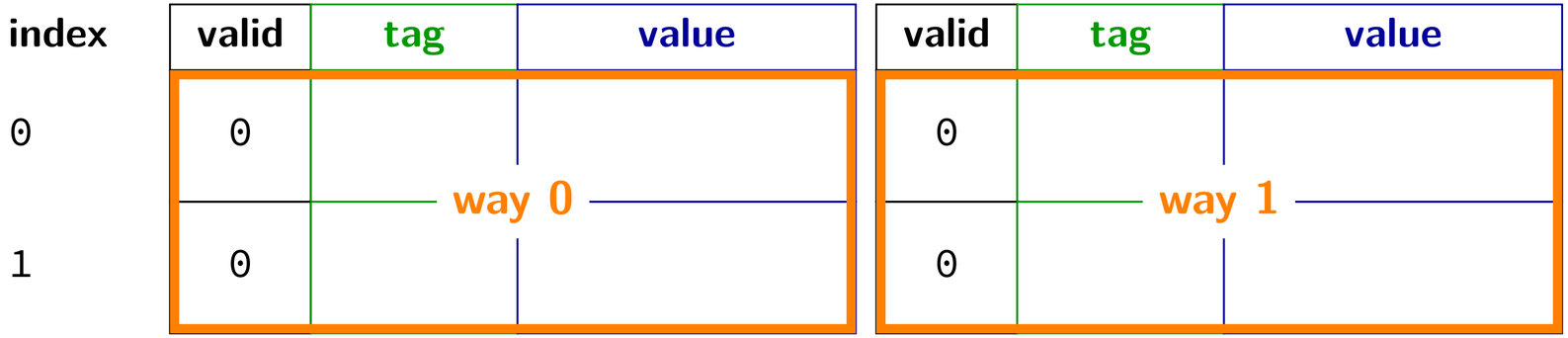
adding associativity

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	0		set 0	0		
1	0		set 1	0		

adding associativity

2-way set associative, 2 byte blocks, 2 sets



adding associativity

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	0			0		
1	0			0		

$m = 8$ bit addresses
$S = 2 = 2^s$ sets
$s = 1$ (set) index bits

$B = 2 = 2^b$ byte block size
$b = 1$ (block) offset bits
$t = m - (s + b) = 6$ tag bits

associative lookup possibilities

none of the blocks for the index are valid

none of the valid blocks for the index match the tag

- something else is stored there

one of the blocks for the index is valid and matches the tag

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0						
1						

address (as hex)	result
00000000(00)	
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	*1*	*000000*	*mem[0x00]* *mem[0x01]*			
1						

address (as hex)	result
00000000(00)	*miss*
00000001(01)	
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]			
1						

address (as hex)	result
00000000(00)	miss
00000001(01)	*hit*
01100011(63)	
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]			
1	*1*	*011000*	*mem[0x62]* *mem[0x63]*			

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	*miss*
01100001(61)	
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]	*1*	*011000*	*mem[0x60]* *mem[0x61]*
1	1	011000	mem[0x62] mem[0x63]			

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	*miss*
01100010(62)	
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]
1	1	011000	mem[0x62] mem[0x63]			

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	*hit*
00000000(00)	
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]
1	1	011000	mem[0x62] mem[0x63]			

address (as hex)	result
00000000(00)	miss
00000001(01)	hit
01100011(63)	miss
01100001(61)	miss
01100010(62)	hit
00000000(00)	*hit*
01100100(64)	

tag index offset

associativity accesses example (steps 1-7)

2-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]
1	1	011000	mem[0x62] mem[0x63]			

address (as hex)	result
00000000(00)	miss
00000001(01)	hit

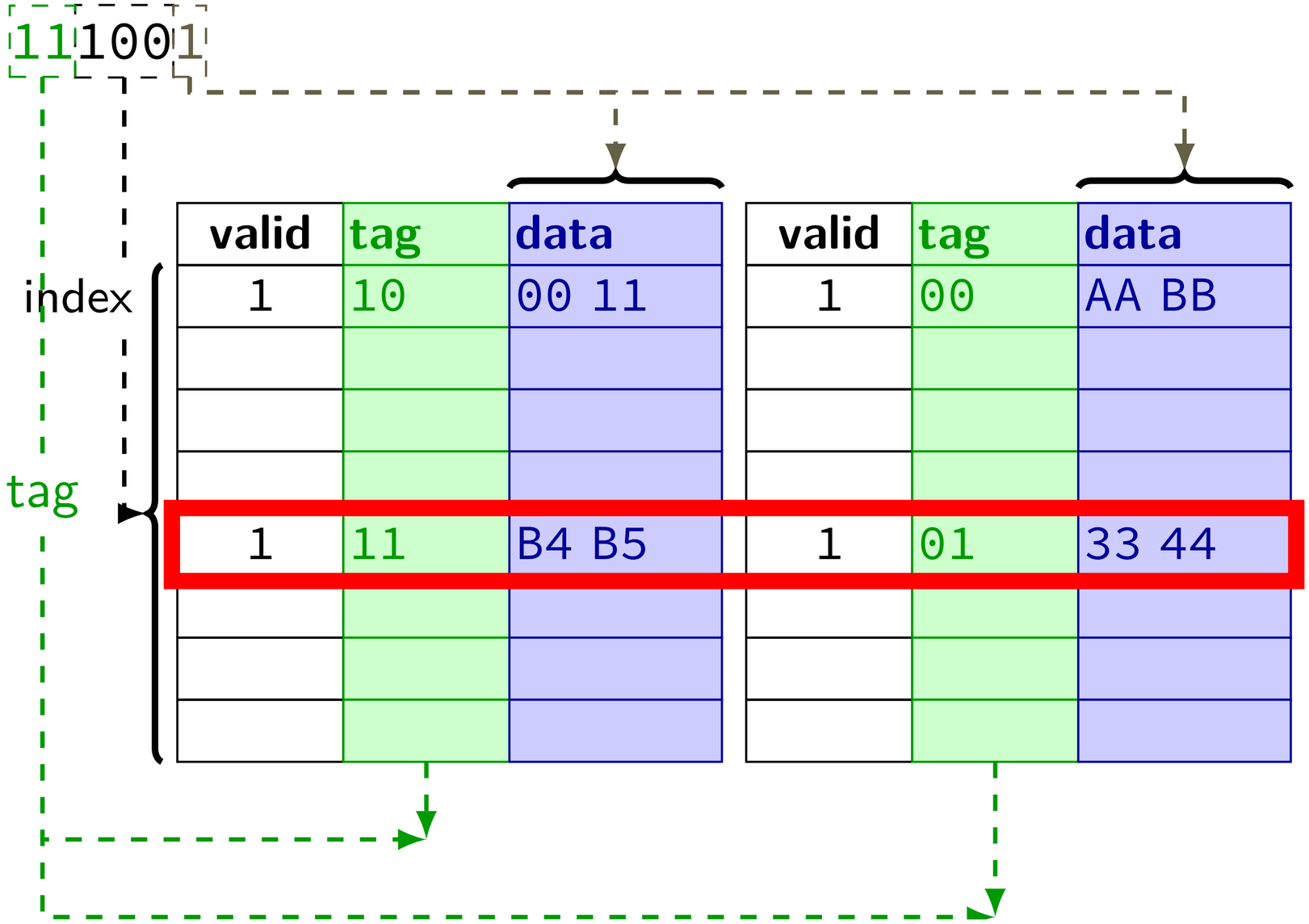
for 0x64, need to replace something in set 0

question: which way?

00000000(00)	hit
01100100(64)	*miss*

tag | index | offset

cache operation (associative)



least recently used

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]
1	1	011000	mem[0x62] mem[0x63]	0		

how to decide where to insert 0x64?

address (hex)	result
00000000 (0	
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	hit
01100100 (64)	miss

least recently used

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]	1
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
00000000 (00)	miss
00000001 (01)	hit
01100011 (63)	miss
01100001 (61)	miss
01100010 (62)	hit
00000000 (00)	hit
01100100 (64)	miss

track which block was read least recently updated on *every access*

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value
0	1	000000	mem[0x00] mem[0x01]			
1	1	011000	mem[0x62] mem[0x63]	0		

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	
01100000 (61)	
00000000 (00)	
11110000 (f0)	
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	*1*	*011000*	*mem[0x60]* *mem[0x61]*	*0*
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	
01100000 (61)	
00000000 (00)	
11110000 (f0)	
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]	0
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	
11110000 (f0)	
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]	*1*
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	011000	mem[0x60] mem[0x61]	1
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	miss
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	*111100*	*mem[0xf0]* *mem[0xf1]*	*0*
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	miss
11100000 (e0)	

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	000000	mem[0x00] mem[0x01]	1	111100	mem[0xf0] mem[0xf1]	0
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	miss
11100000 (e0)	miss

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	*1*	*111000*	*mem[0xe0]* *mem[0xe1]*	1	111100	mem[0xf0] mem[0xf1]	*1*
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	miss
11100000 (e0)	miss

LRU bit updating

2-way set associative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	LRU
0	1	111000	mem[0xe0] mem[0xe1]	1	111100	mem[0xf0] mem[0xf1]	1
1	1	011000	mem[0x62] mem[0x63]	0			1

address (hex)	result
...	...
01100001 (61)	miss
01100000 (60)	hit
01100000 (61)	hit
00000000 (00)	hit
11110000 (f0)	miss
11100000 (e0)	miss

LRU w/ more than two ways?

need to track total order

worst case: bunch of new accesses to same set

- first replaces least recently used
- next replaces next least recently used
- etc.

3-way set associtiative, 2 byte blocks, 2 sets

index	valid	tag	value	valid	tag	value	valid	tag	value	LRU
0	1	…	…	1	…	…	1	…	…	0 then 1
0	1	…	…	1	…	…	1	…	…	2 then 0

hard to track, so frequently only approximated

example replacement policies

least recently used

- take advantage of *temporal locality*
- at least $\lceil \log_2(E!) \rceil$ bits per set for E-way cache
  - (need to store order of all blocks)

approximations of least recently used

- implementing least recently used is expensive
- really just need "avoid recently used" — much faster/simpler
- good approximations: E to $2E$ bits

first-in, first-out

- counter per set — where to replace next

(pseudo-)random

- no extra information!
- actually works pretty well in practice

associativity terminology

direct-mapped — one block per set

E-way set associative — E blocks per set

E ways in the cache

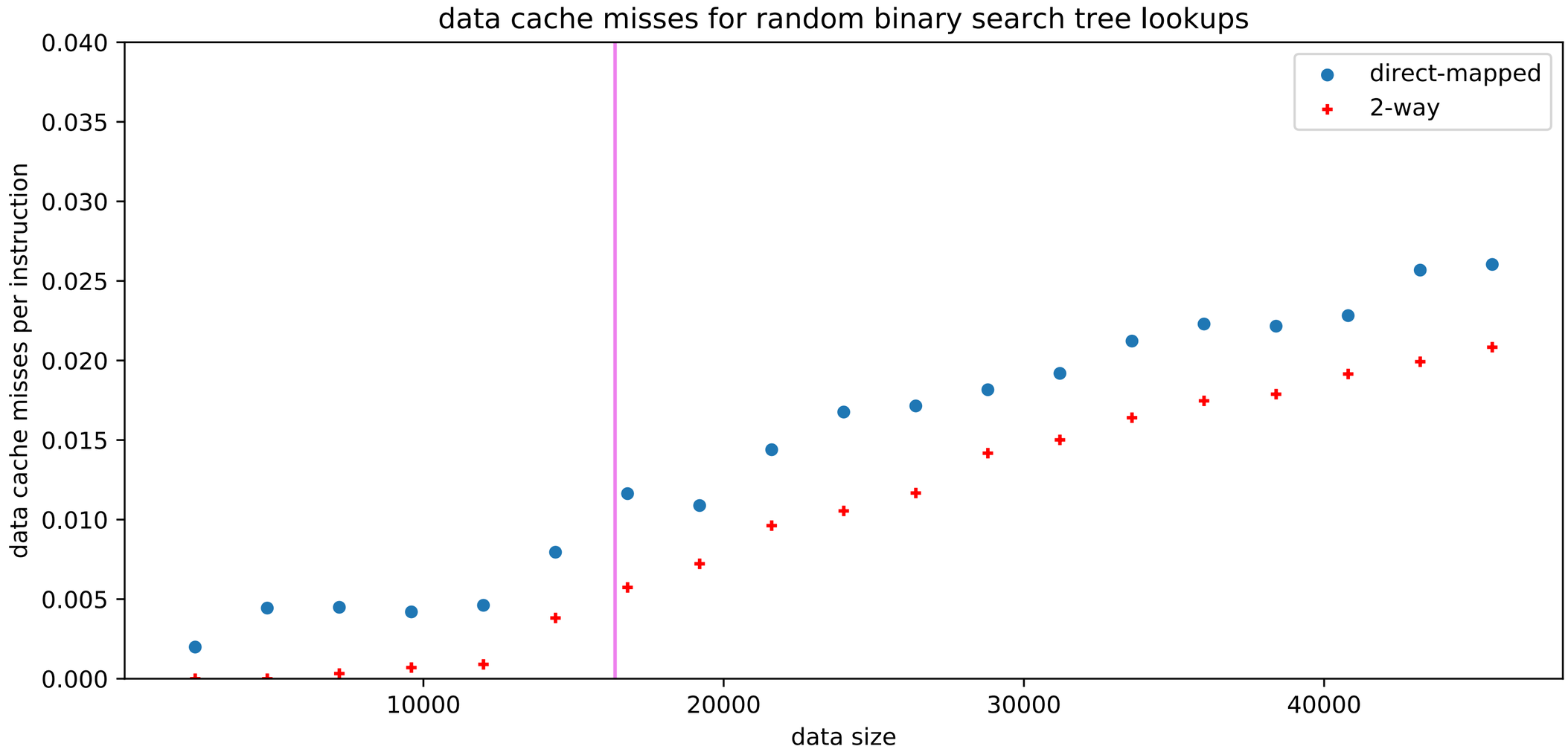
fully associative — one set total (everything in one set)

Tag-Index-Offset formulas

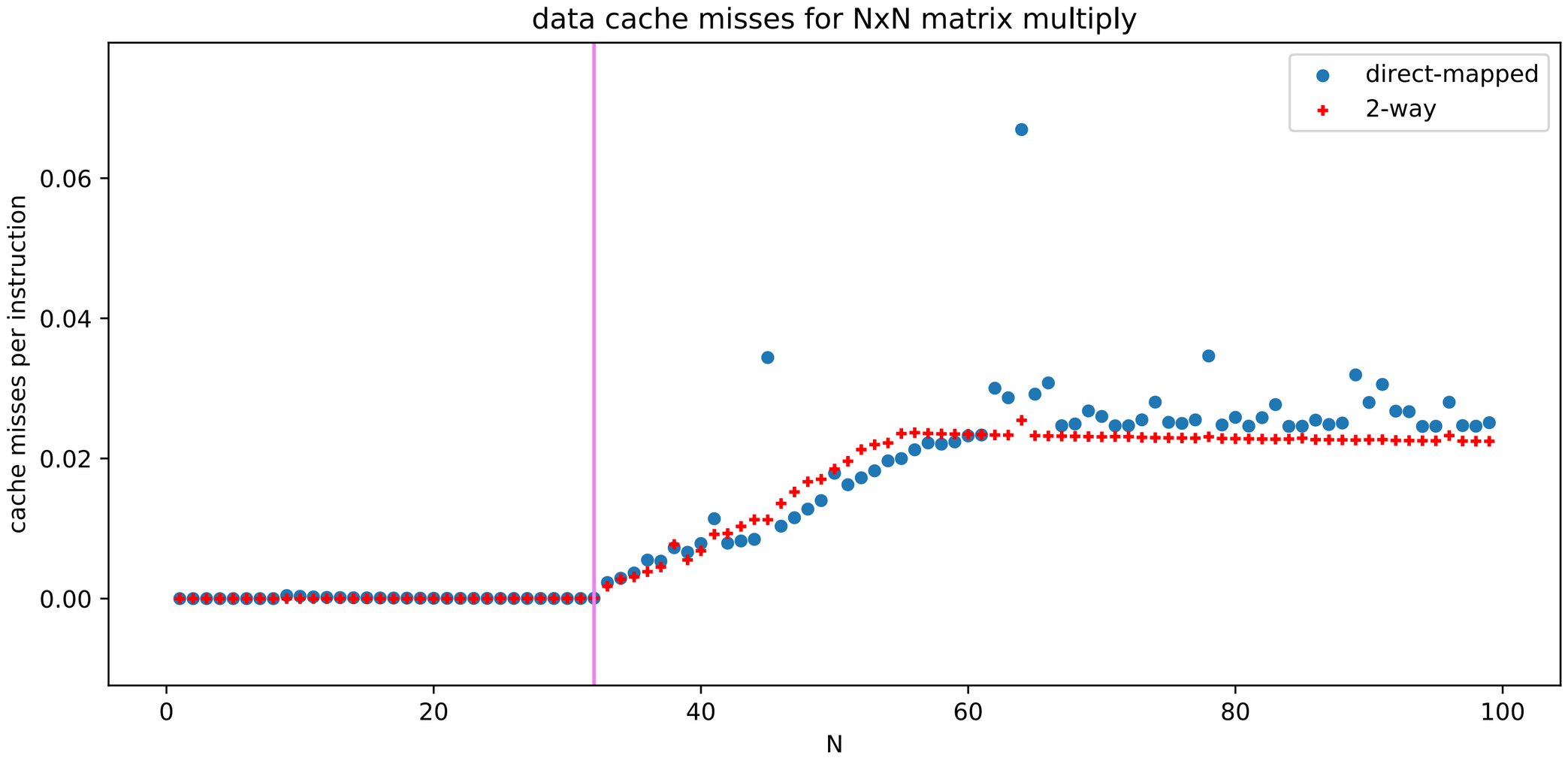
m	memory addreses bits
E	number of blocks per set (“ways”)
$S = 2^s$	number of sets
s	(set) index bits
$B = 2^b$	block size
b	(block) offset bits
$t = m - (s + b)$	tag bits
$C = B \times S \times E$	cache size (excluding metadata)

backup slides

simulated misses: BST lookups



simulated misses: matrix multiplies



Tag-Index-Offset exercise

m	memory addreses bits (Y86-64: 64)
E	number of blocks per set (“ways”)
$S = 2^s$	number of sets
s	(set) index bits
$B = 2^b$	block size
b	(block) offset bits
$t = m - (s + b)$	tag bits
$C = B \times S \times E$	cache size (excluding metadata)

My desktop:

- L1 Data Cache: 32 KB, 8 blocks/set, 64 byte blocks
- L2 Cache: 256 KB, 4 blocks/set, 64 byte blocks
- L3 Cache: 8 MB, 16 blocks/set, 64 byte blocks
- Divide the address `0x34567` into *tag*, *index*, *offset* for each cache.

T-I-O exercise: L1

quantity	value for L1
block size (given)	$B = 64\text{Byte}$
$B = 2^b$ (b: block offset bits)	
block offset bits	$b = 6$
blocks/set (given)	$E = 8$
cache size (given)	$C = 32\text{KB} = E \times B \times S$
$S = \frac{C}{B \times E}$ (S: number of sets)	
number of sets	$S = \frac{32\text{KB}}{64\text{Byte} \times 8} = 64$
$S = 2^s$ (s: set index bits)	
set index bits	$s = \log_2(64) = 6$

T-I-O results

	L1	L2	L3
sets	64	1024	8192
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-5 (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	*6*	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-*5* (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3     4     5     6     7
          0011  0100  0101  0110  0111
```

bits 0-5 (all offsets): *`100111`* = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-5 (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-5 (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): *0x34*

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-5 (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

T-I-O: splitting

	L1	L2	L3
block offset bits	6	6	6
set index bits	6	10	13
tag bits	(the rest)		

`0x34567`:

```
          3    4    5    6    7
          0011 0100 0101 0110 0111
```

bits 0-5 (all offsets): `100111` = `0x27`

L1:

- bits 6-11 (L1 set): `01 0101` = `0x15`
- bits 12- (L1 tag): 0x34

L2:

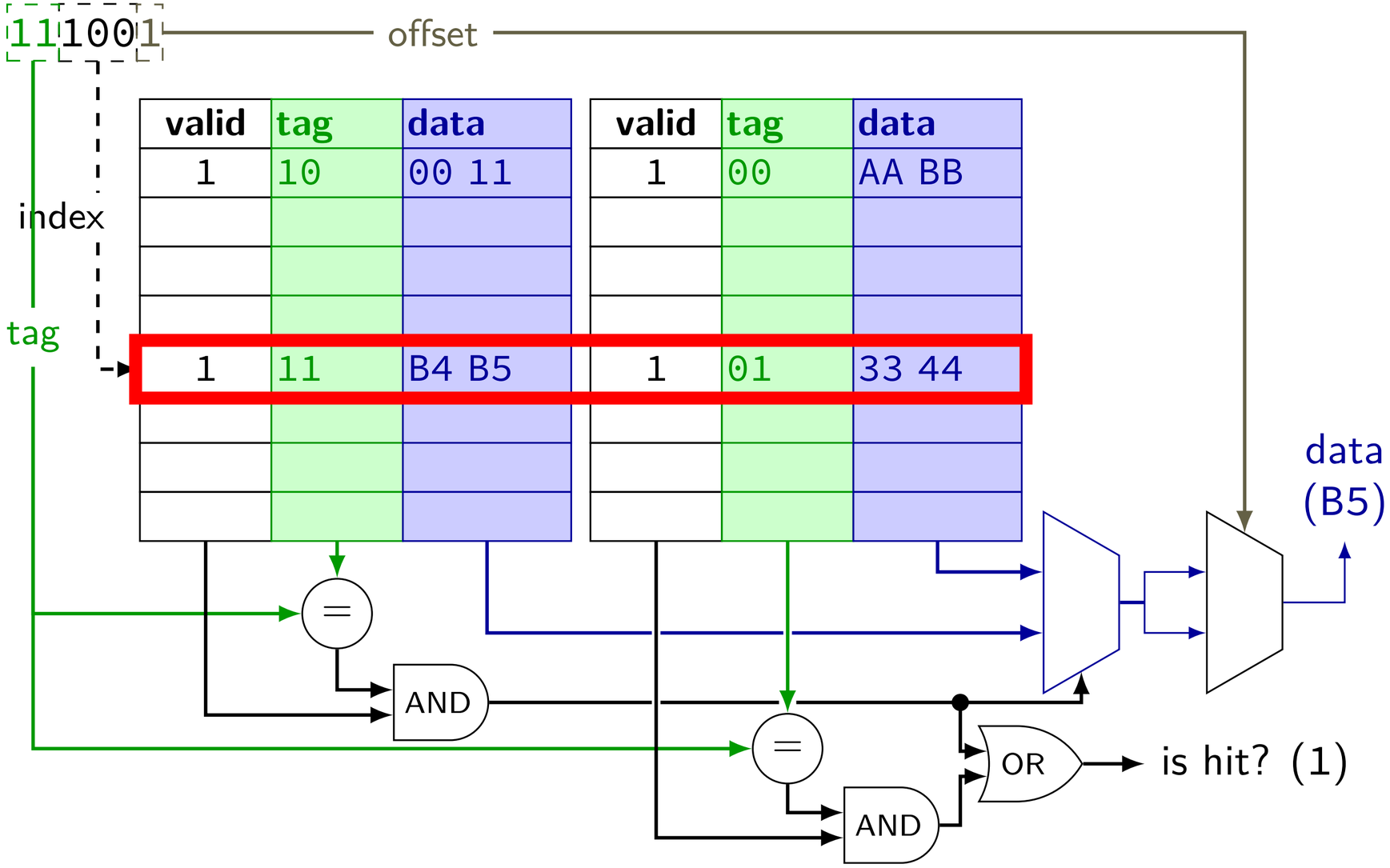
- bits 6-15 (set for L2): `01 0001 0101` = `0x115`
- bits 16-: `0x3`

L3:

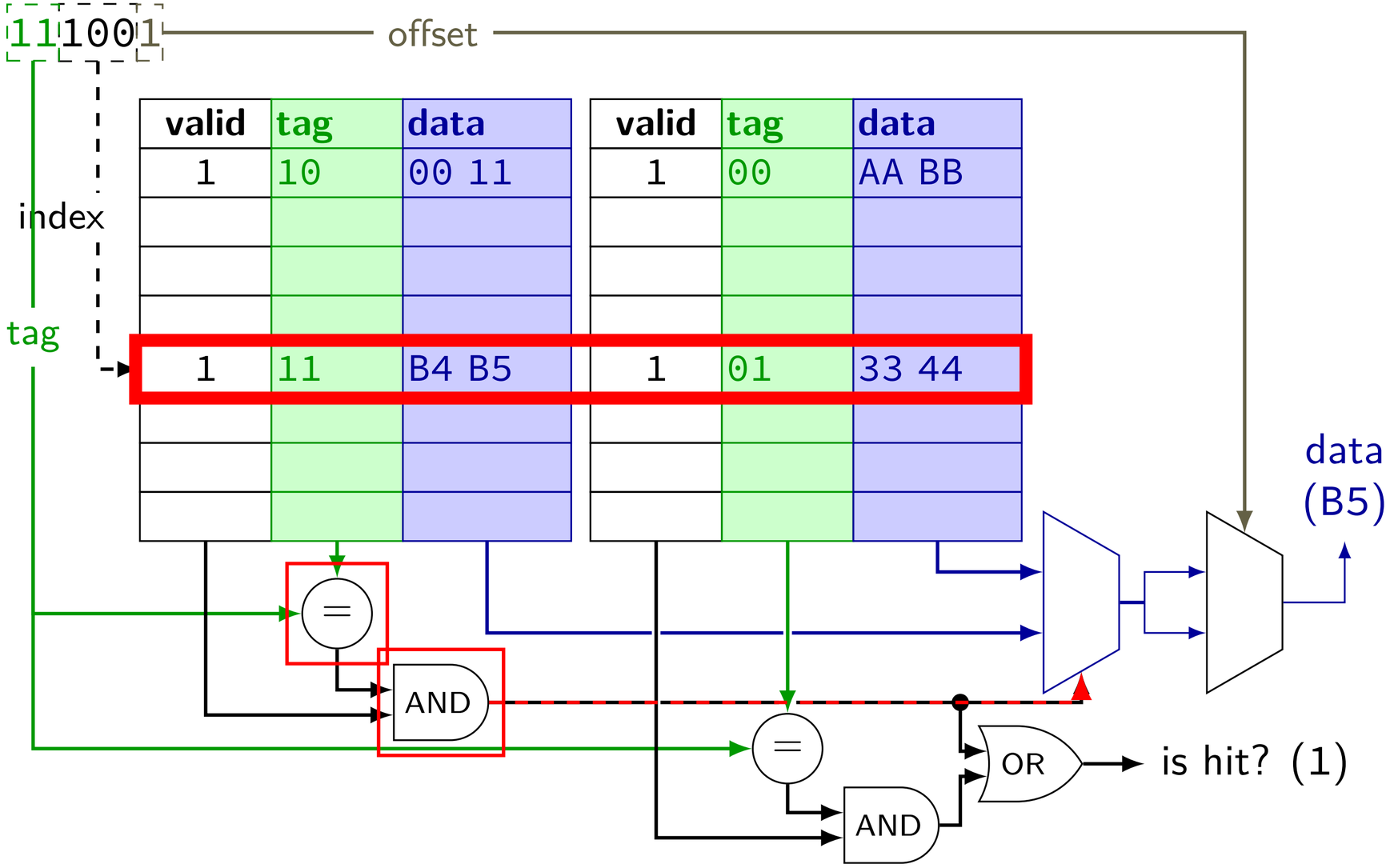
- bits 6-18 (set for L3): `0 1101 0001 0101` = `0xD15`
- bits 18-: `0x0`

cache operation (associative)

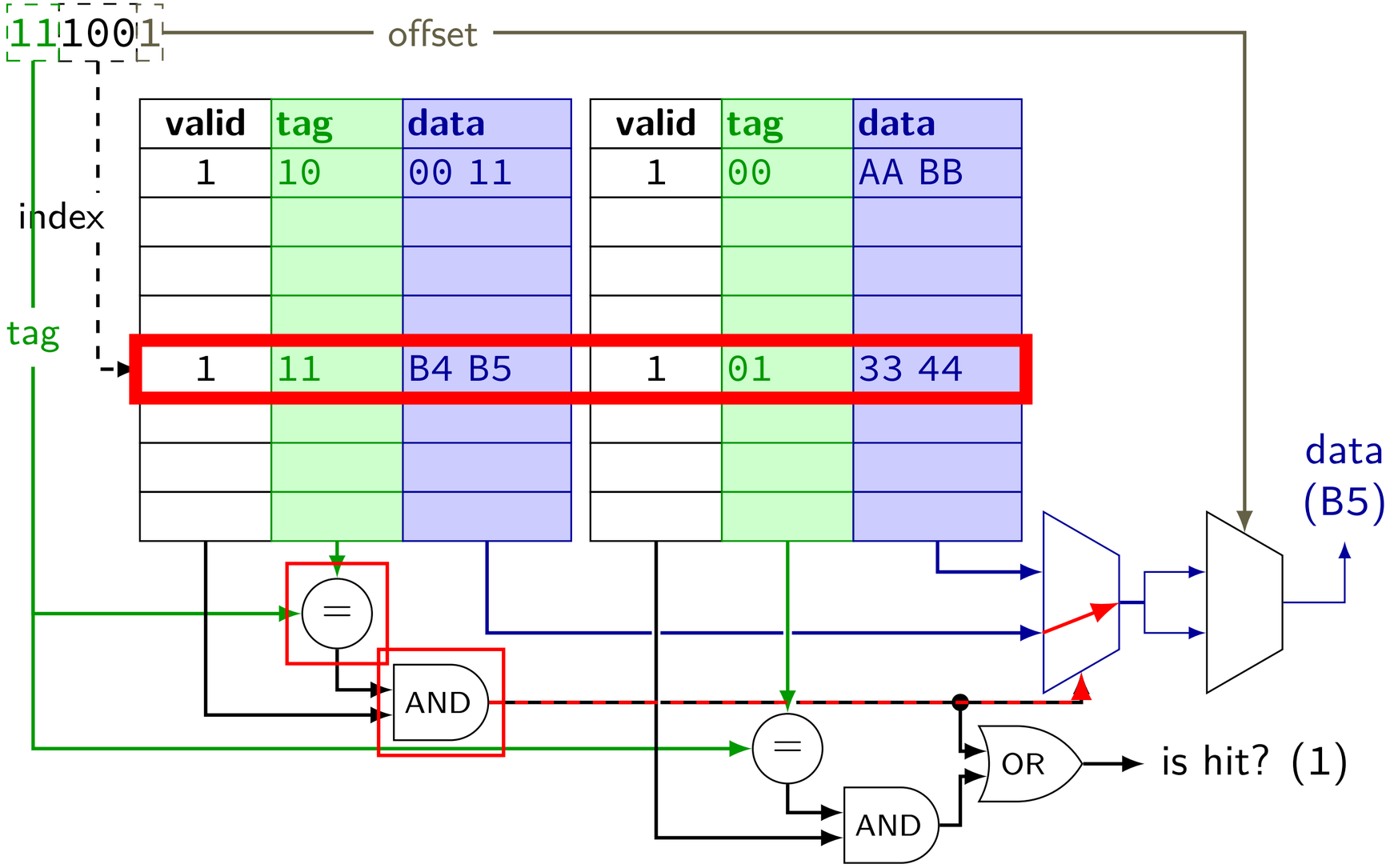
cache operation (associative)



cache operation (associative)

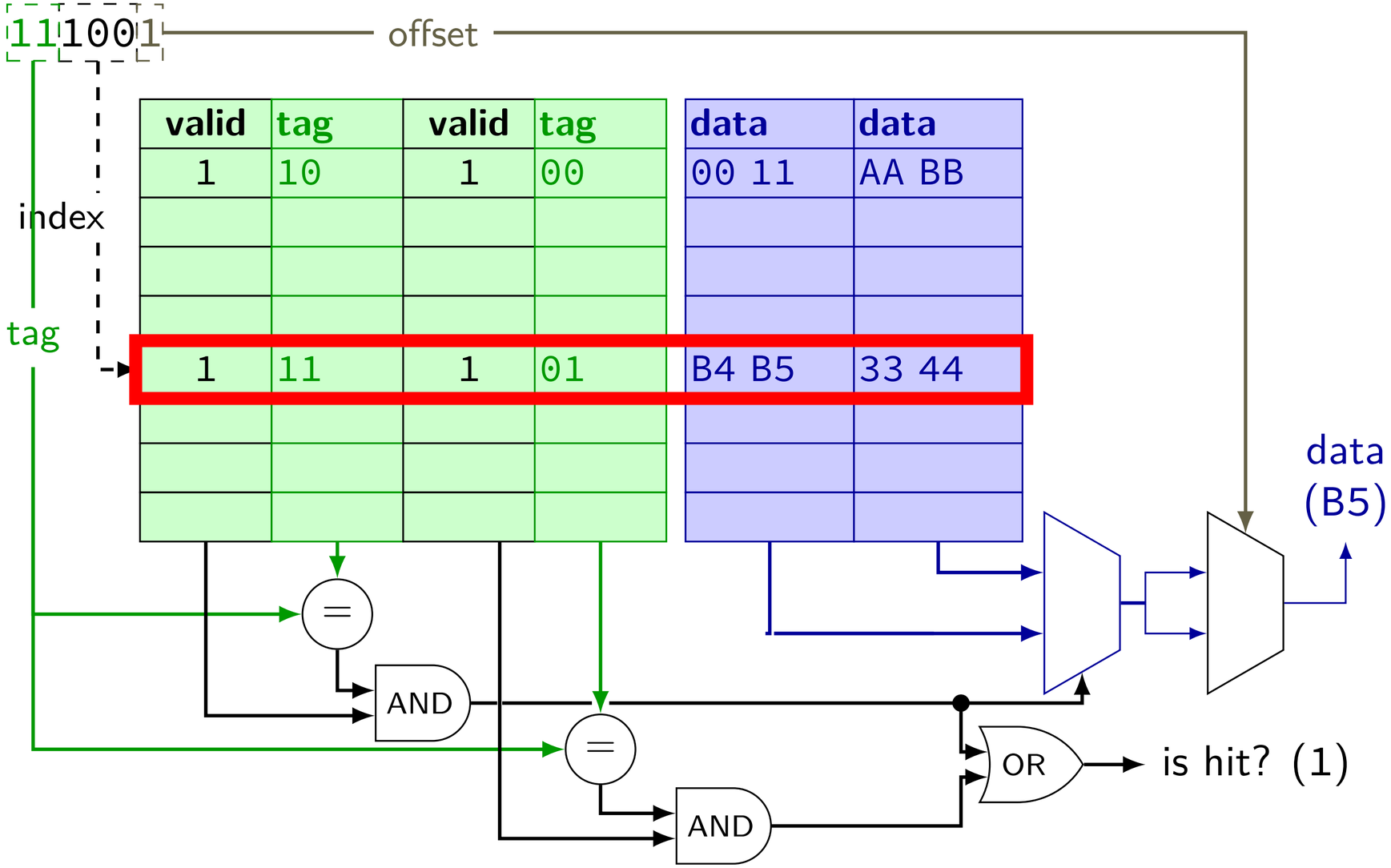


cache operation (associative)

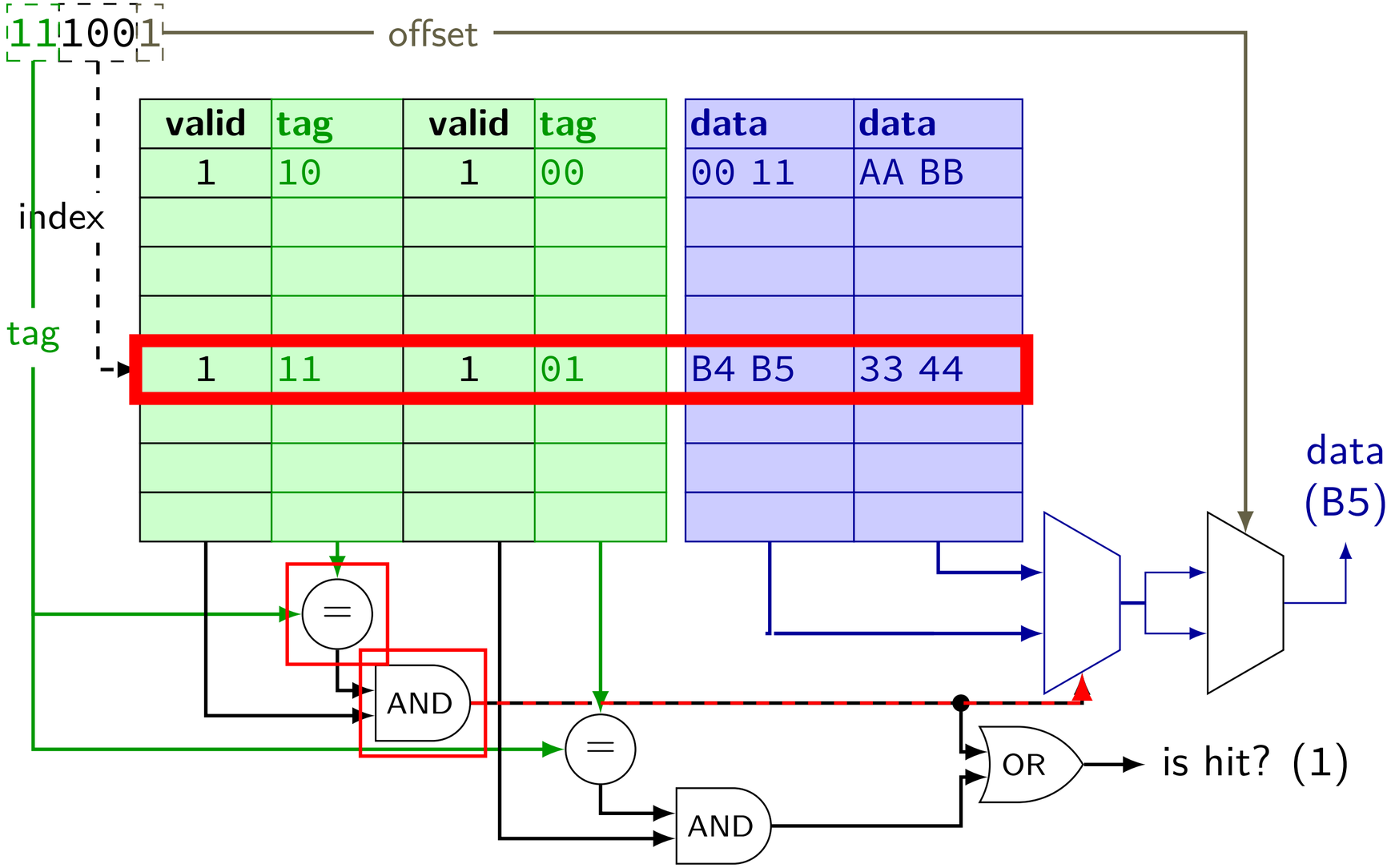


cache operation (associative)

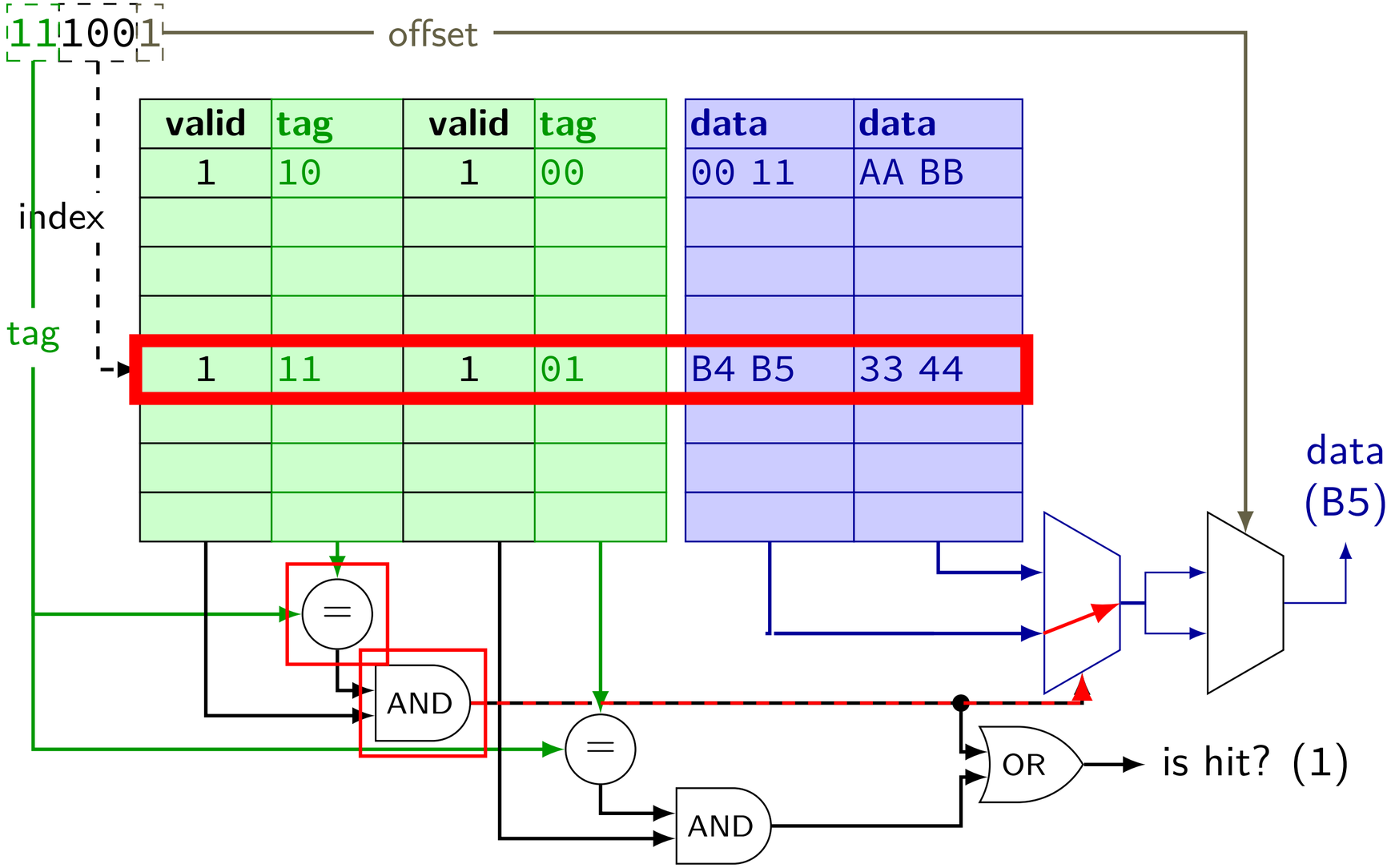
cache operation (associative)



cache operation (associative)



cache operation (associative)



exercise

exercise

index	V	tag	value
0	0	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	06-78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value
06-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-
0		

LRU
ay 06ay
17ay 08<9>v 19

address (hex)	hit?
00000000 (00)	
00000001 (01)	it6-
00001010 (0A)	
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

exercise

index	V	tag	value	V	tag	value	LRU
0	0	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]	06-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-	ay 06ay
1	06- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-	0			17ay 08 18- 08<9>v 19

address (hex)	hit?
00000000 (00)	
00000001 (01)	it6-
00001010 (0A)	
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

how is the address 21 (00100001) split up into tag/index/offset?

b block offset bits;
$B = 2^b$ byte block size;
s set index bits; $S = 2^s$ sets;
$t = m - (s + b)$ tag bits (leftover)

exercise

index	V	tag	value
0	0	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	06-78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value
06-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-
0		

LRU
ay 06ay
17ay 08 18- 19

address (hex)	hit?
00000000 (00)	
00000001 (01)	it6-
00001010 (0A)	
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

tag index offset

exercise

index	V	tag	value	V	tag	value	LRU
0	0	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]	06-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-	ay 06ay
1	06- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-	0			17ay 08 18- 08<9>v 19

address (hex)	hit?
00000000 (00)	
00000001 (01)	it6-
00001010 (0A)	
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

tag index offset

exercise: how many accesses are hits?
what is the final state of the cache?

exercise

index	V	tag	value	V	tag	value	LRU
0	1	0000000006<7>001007-	M[0x00] M[0x01] M[0x02] M[0x03][0x00]	06-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-	way 1ay
1	06- 78-	00008-	M[0x01] M[0x02] [0x0C] M[0x0D] M[0x03]6[0x20] M[0x0E] M[0x0F]8- M[0x21] M[0x22] M[0x23]7-	0			06ay 17ay 08<9>v 19

address (hex)	hit?
00000000 (00)	miss
00000001 (01)	it6-
00001010 (0A)	
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

tag index offset

exercise: how many accesses are hits?
what is the final state of the cache?

exercise

index	V	tag	value
0	1	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	6- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value	LRU
6-	00016- 7<8>000008-	M[0x08] M[0x09] M[0x0A] M[0x0B][0x00] M[0x01]	ay 06ay
0		M[0x02] M[0x03]8-	17ay 08<9> 19

address (hex)	hit?
00000000 (00)	miss
00000001 (01)	it6-
00001010 (0A)	iss6-
00100001 (21)	
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

tag index offset

exercise

index	V	tag	value
0	1	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	6- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value	LRU
6-	00016- 7<8>000008-	M[0x08] M[0x09] M[0x0A] M[0x0B][0x00] M[0x01] M[0x02] M[0x03]8-	ay 06ay
0			17ay 08<9> 19

address (hex)	hit?
00000000 (00)	miss
00000001 (01)	it6-
00001010 (0A)	iss6-
00100001 (21)	iss7-
00001100 (0C)	iss7-
00000011 (02)	
00100011 (23)	

tag index offset

exercise

index	V	tag	value
0	1	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	6- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value
6-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-
0		

LRU
ay 06ay
17ay 08 18- 08<9>v 19

address (hex)	hit?
00000000 (00)	miss
00000001 (01)	it6-
00001010 (0A)	iss6-
00100001 (21)	iss7-
00001100 (0C)	iss7-
00000011 (02)	iss8-
00100011 (23)	

tag index offset

exercise

index	V	tag	value
0	1	00006<7>001007-	[0x00] M[0x01] M[0x02] M[0x03]6[0x20]
1	6- 78-	00008-	[0x0C] M[0x0D] M[0x21] M[0x22] M[0x0F] M[0x0F]8- M[0x23]7-

V	tag	value
6-	00016- 7<8>000008-	[0x00] M[0x01] M[0x02] M[0x03]8-
0		

LRU
ay 06ay
17ay 08 18- 08<9>v 19

address (hex)	hit?
00000000 (00)	miss
00000001 (01)	it6-
00001010 (0A)	iss6-
00100001 (21)	iss7-
00001100 (0C)	iss7-
00000011 (02)	iss8-
00100011 (23)	it9-

tag index offset

exercise (1)

1 set, 5-way cache, 4 byte blocks

access pattern:

0x0, 0x1000, 0x4, 0x1004, 0x0, 0x1008, 0x4, 0x100C,
0x0, 0x1010, 0x4, 0x1014, 0x0, 0x1018, 0x4, 0x101C,
0x0, 0x1020, 0x4, 0x1024, 0x0, 0x1028, 0x4, 0x102C,
…

how does each do?

LRU
FIFO
random

LRU

access MRU $\leftarrow\rightarrow$ LRU *0x0 *0x0-3* 0x1000 *0x1000-3*, 0x0-3 *0x4 *0x4-7*, 0x1000-3, 0x0-3 0x1004 *0x1004-7*, 0x4-7, 0x1000-3, 0x0-3 0x0 0x0-3, 0x1004-7, 0x4-7, 0x1000-3 0x1008 *0x1008-B*, 0x0-3, 0x1004-7, 0x4-7, 0x1000-3 0x4 0x4-7, 0x1008-B, 0x0-3, 0x1004-7, 0x1000-3 0x100C *0x100C-F*, 0x4-7, 0x1008-B, 0x0-3, 0x1004-7 0x0 0x0-3, 0x100C-F, 0x4-7, 0x1008-B, 0x1004-7 0x1010 *0x1010-3*, 0x0-3, 0x100C-F, 0x4-7, 0x1008-B … … others: 0% hit rate

0x0/0x4: 100% hit rate (after ‘warmup’)

FIFO (1)

access	last in $leftarrow$ $rightarrow$ first in
0x0	*0x0-3*
0x1000	*0x1000-3*, 0x0-3
0x4	*0x4-7*, 0x1000-3, 0x0-3
0x1004	*0x1004-7*, 0x4-7, 0x1000-3, 0x0-3
0x0	0x1004-7, 0x4-7, 0x1000-3, 0x0-3
0x1008	*0x1008-B*, 0x1004-7, 0x4-7, 0x1000-3, 0x0-3
0x4	0x1008-B, 0x1004-7, 0x4-7, 0x1000-3, 0x0-3
0x100C	*0x100C-F*, 0x1008-B, 0x1004-7, 0x4-7, 0x1000-3
0x0	*0x0-3*, 0x100C-F, 0x1008-B, 0x1004-7, 0x4-7
0x1010	*0x1010-3*, 0x0-3, 0x100C-F, 0x1008-B, 0x1004-7

FIFO (2)

access	last in $leftarrow$ $rightarrow$ first in
…	…
0x100C	*0x100C-F*, 0x1008-B, 0x1004-7, 0x4-7, 0x1000-3
0x0	*0x0-3*, 0x100C-F, 0x1008-B, 0x1004-7, 0x4-7
0x1010	*0x1010-3*, 0x0-3, 0x100C-F, 0x1008-B, 0x1004-7
0x4	*0x4-7*, 0x1010-3, 0x0-3, 0x100C-F, 0x1008-B
0x1014	*0x1014-7*, 0x4-7, 0x1010-3, 0x0-3, 0x100C-F
0x0	0x1014-7, 0x4-7, 0x1010-3, 0x0-3, 0x100C-F
0x1018	*0x1018-B*, 0x1014-7, 0x4-7, 0x1010-3, 0x0-3
0x4	0x1018-B, 0x1014-7, 0x4-7, 0x1010-3, 0x0-3
0x101C	*0x101C-F*, 0x1018-B, 0x1014-7, 0x4-7, 0x1010-3

exercise (2)

2 set, 3-way cache, 4 byte blocks

access pattern:

0x0, 0x1000, 0x4, 0x1004, 0x0, 0x1008, 0x4, 0x100C,
0x0, 0x1010, 0x4, 0x1014, 0x0, 0x1018, 0x4, 0x101C,
0x0, 0x1020, 0x4, 0x1024, 0x0, 0x1028, 0x4, 0x102C,
…

how does each do?

LRU
FIFO
random

exercise (2)

2 set, 3-way cache, 4 byte blocks

access pattern:

0x0, 0x1000, 0x4, 0x1004, *0x0, 0x1008,* 0x4, 0x100C,
0x0, 0x1010, 0x4, 0x1014, *0x0, 0x1018,* 0x4, 0x101C,
0x0, 0x1020, 0x4, 0x1024, *0x0, 0x1028,* 0x4, 0x102C,
…

how does each do?

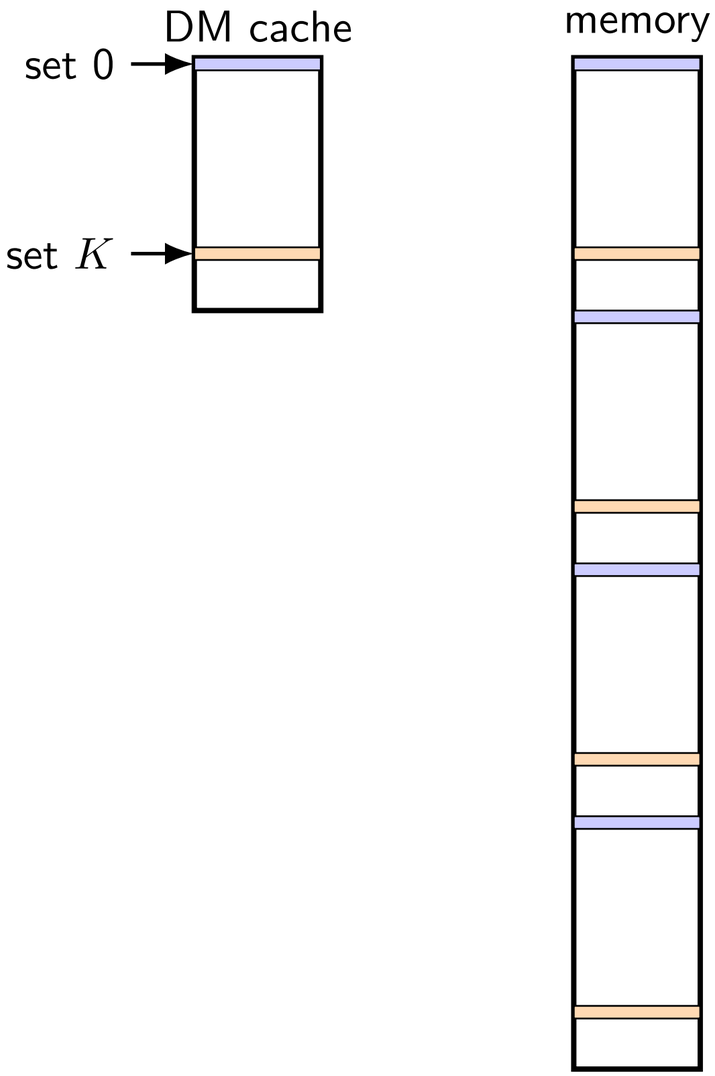
- LRU
- FIFO
- random

FIFO

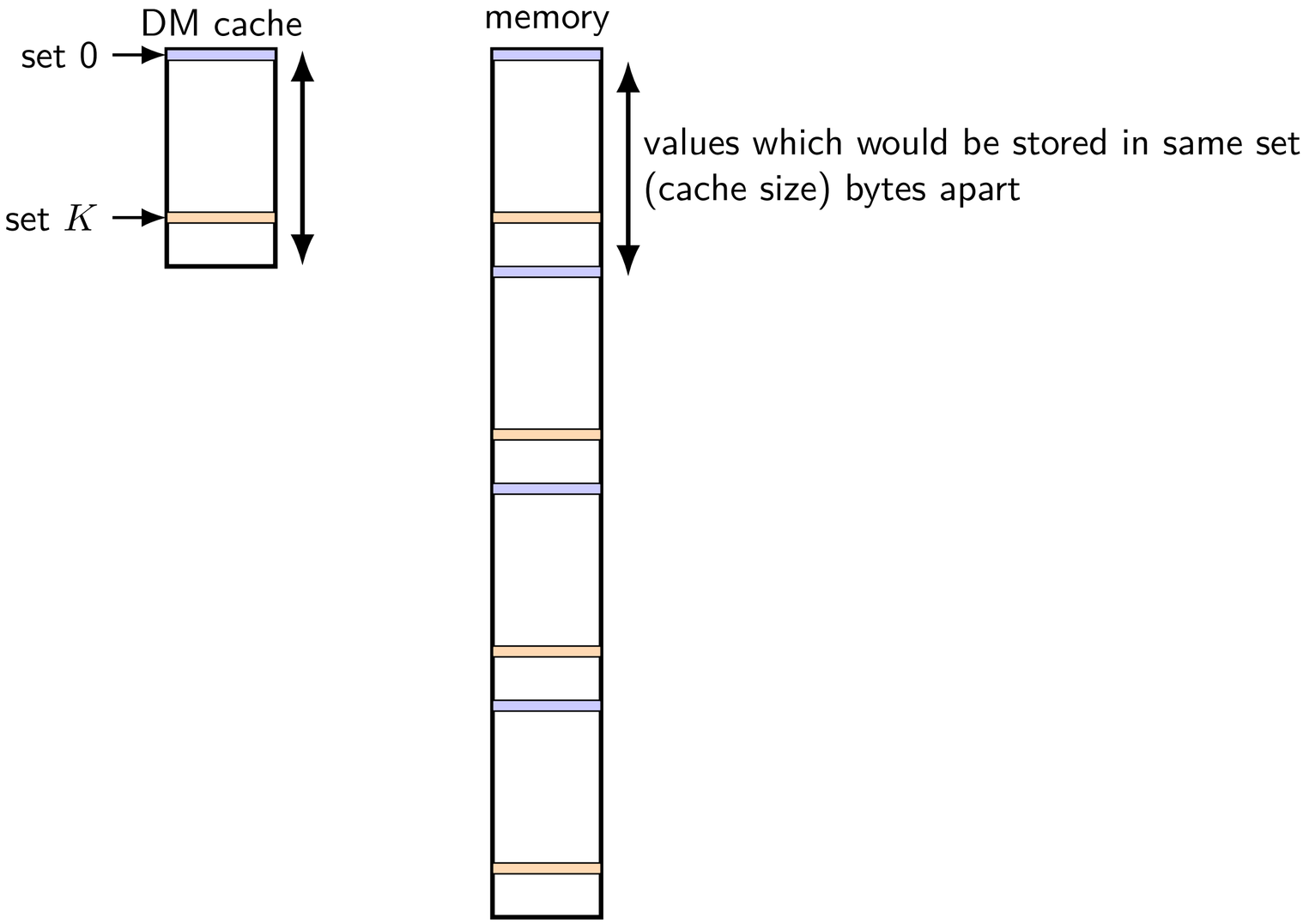
access	last in $leftarrow$ $rightarrow$ first in
0x0	*0x0*
0x1000	*0x1000*, 0x0
0x0	0x1000, 0x0
0x1008	*0x1008*, 0x1000, 0x0
0x0	0x1008, 0x1000, 0x0
0x1010	*0x1010*, 0x1008, 0x1000
0x0	*0x0*, 0x1010, 0x1008
0x1018	*0x1018*, 0x0, 0x1010
0x0	0x1018, 0x0, 0x1010
0x1020	*0x1020*, 0x1018, 0x0

mapping of sets to memory (direct-mapped)

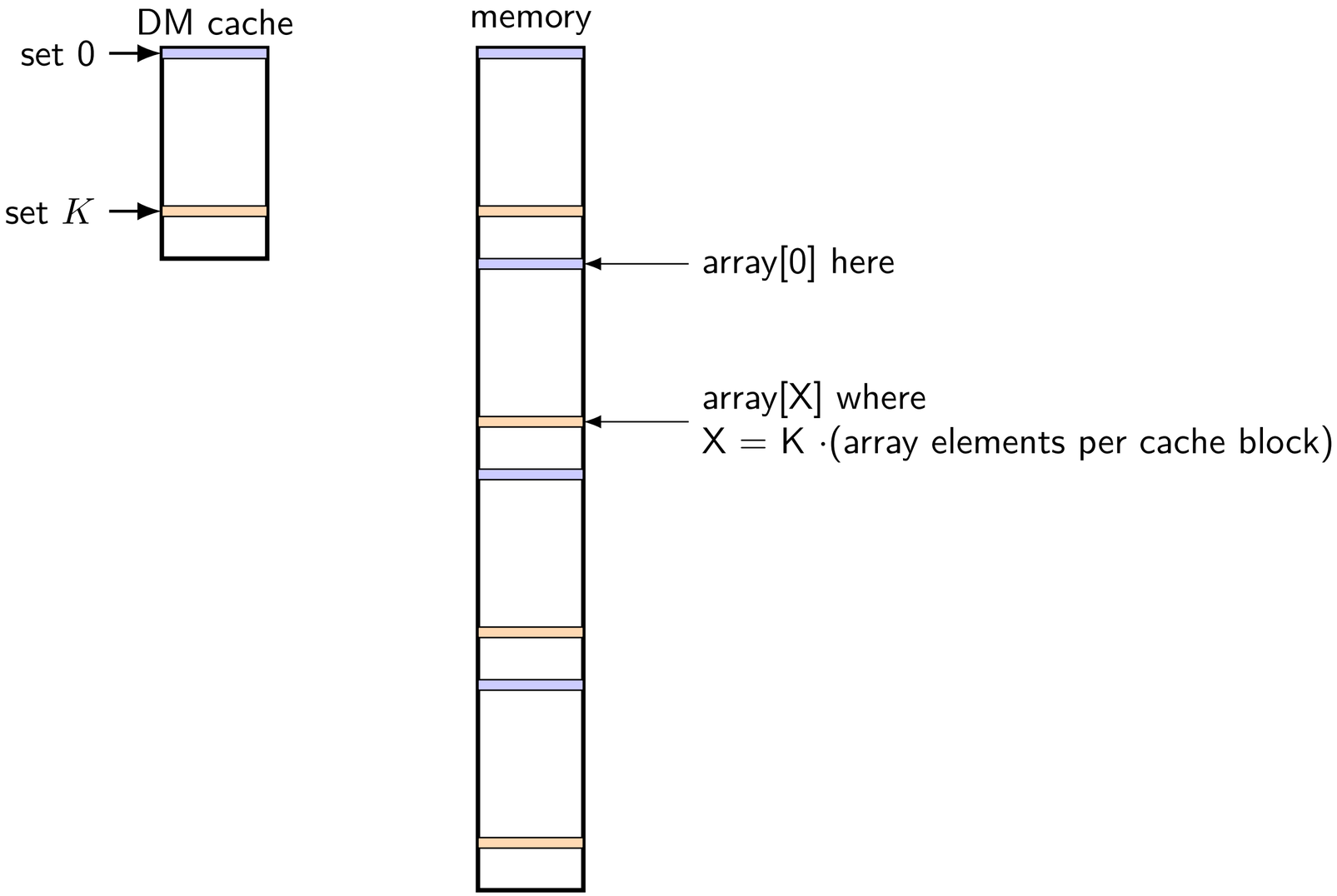
mapping of sets to memory (direct-mapped)



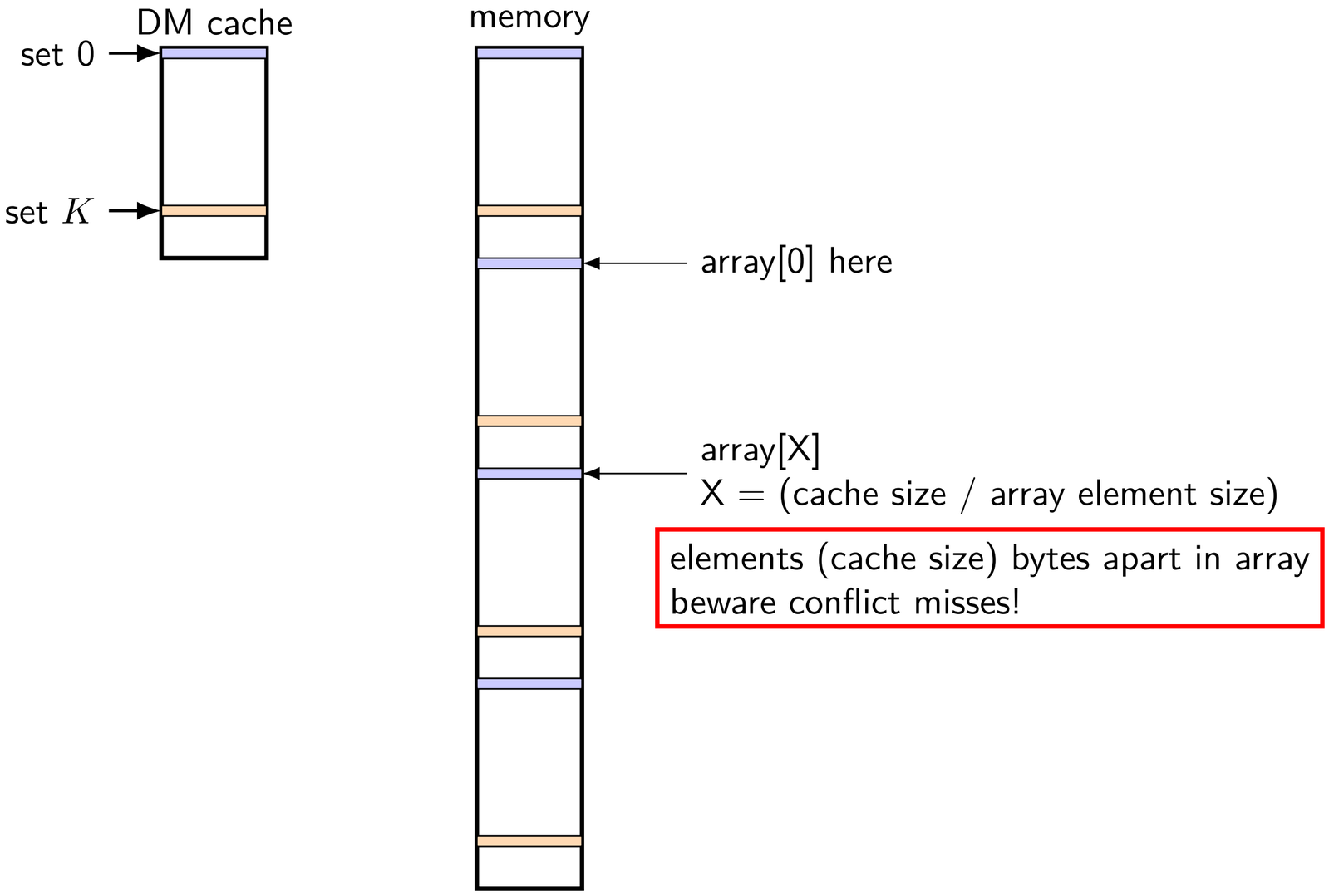
mapping of sets to memory (direct-mapped)



mapping of sets to memory (direct-mapped)

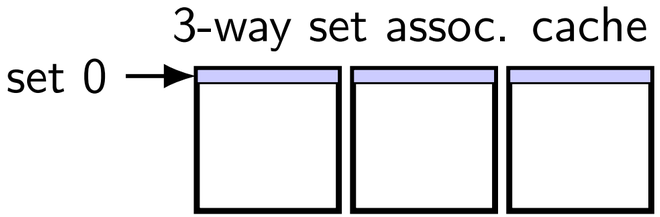


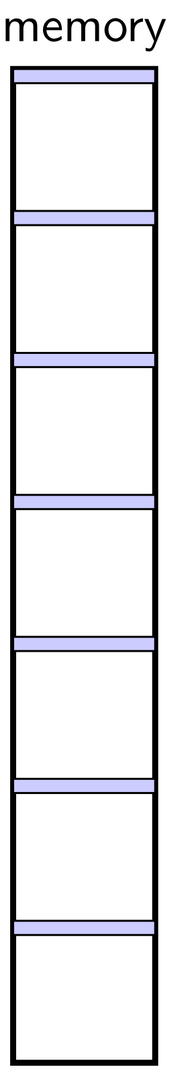
mapping of sets to memory (direct-mapped)



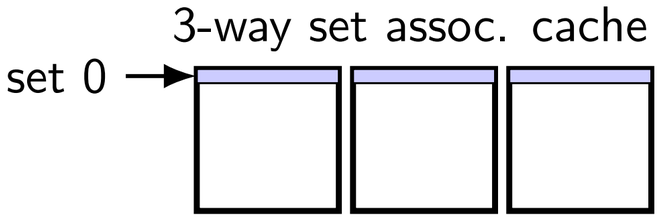
mapping of sets to memory (3-way)

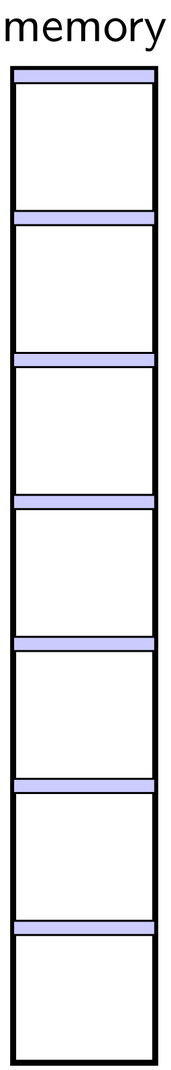
mapping of sets to memory (3-way)



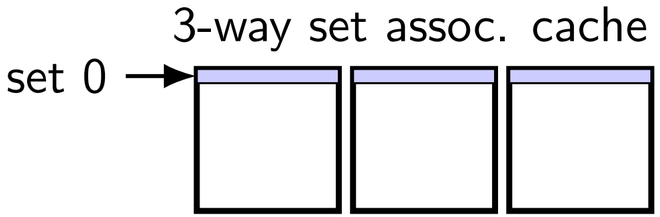


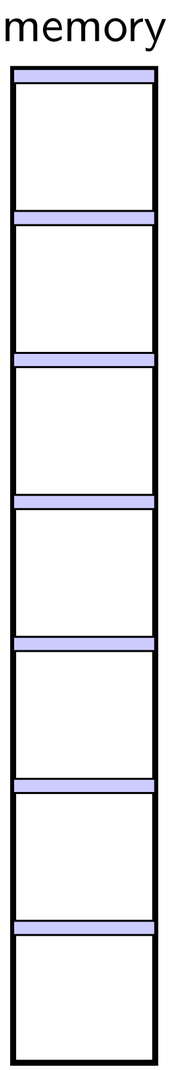
mapping of sets to memory (3-way)



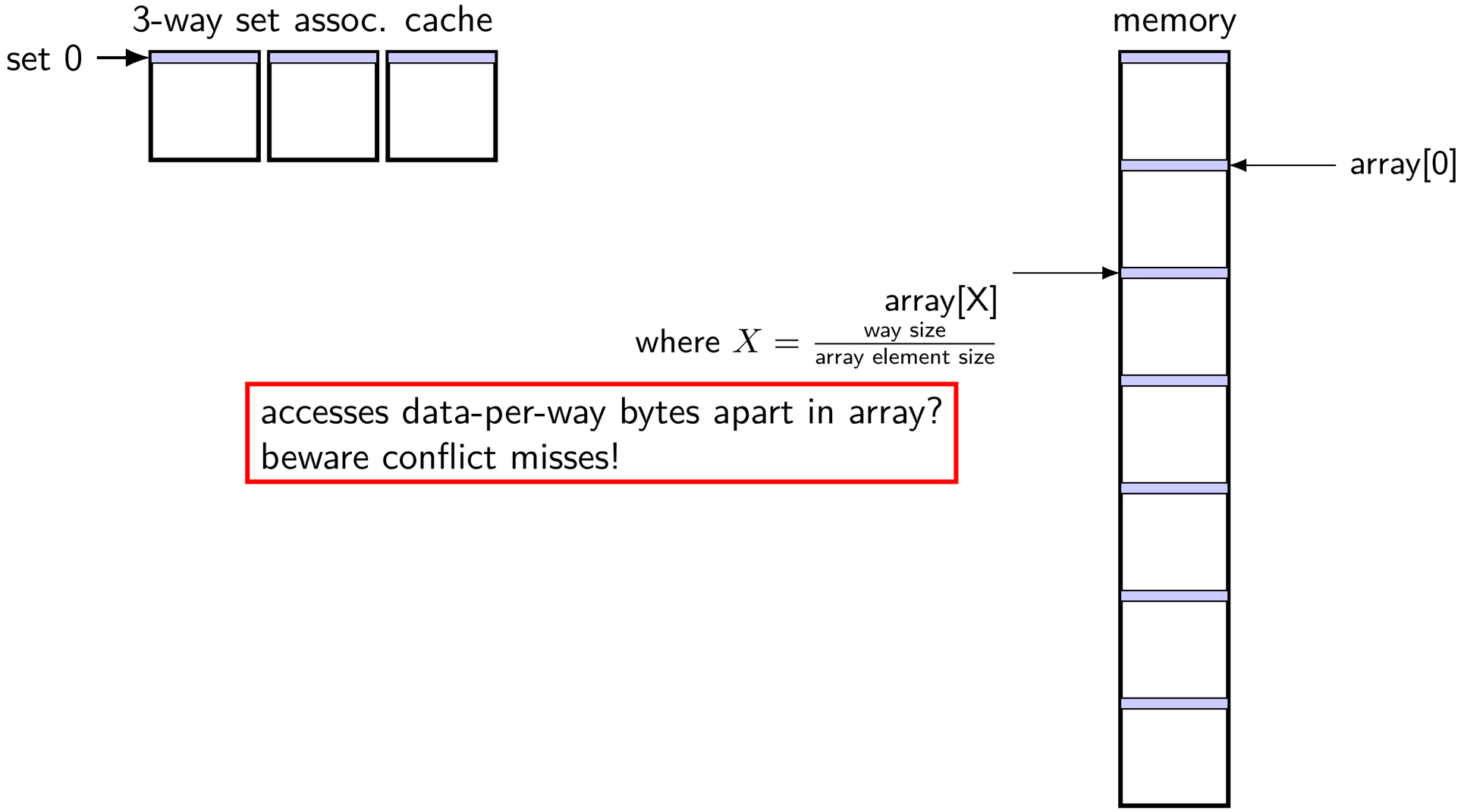


mapping of sets to memory (3-way)





mapping of sets to memory (3-way)



accesses data-per-way bytes apart in array?
beware conflict misses!